Software Industry & Market

12

Lesson

Outline

1. Characteristics of Software
2. Sources of Software Value
3. Viewpoints of SEE Research
4. Paradigm of SEE Analysis
5. Business Models of SEE
6. Law of Software Demand & Supply

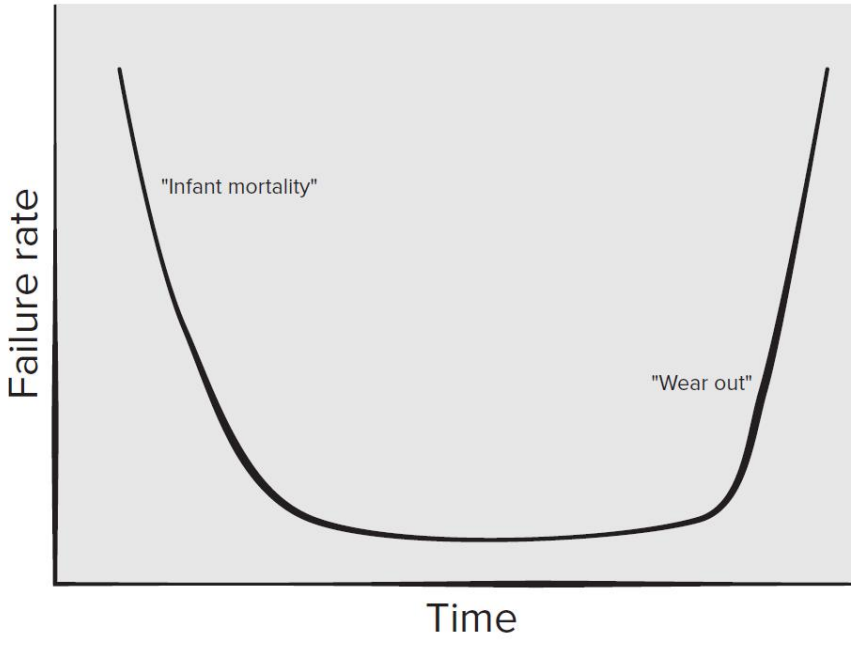
12.1 Characteristics of Software

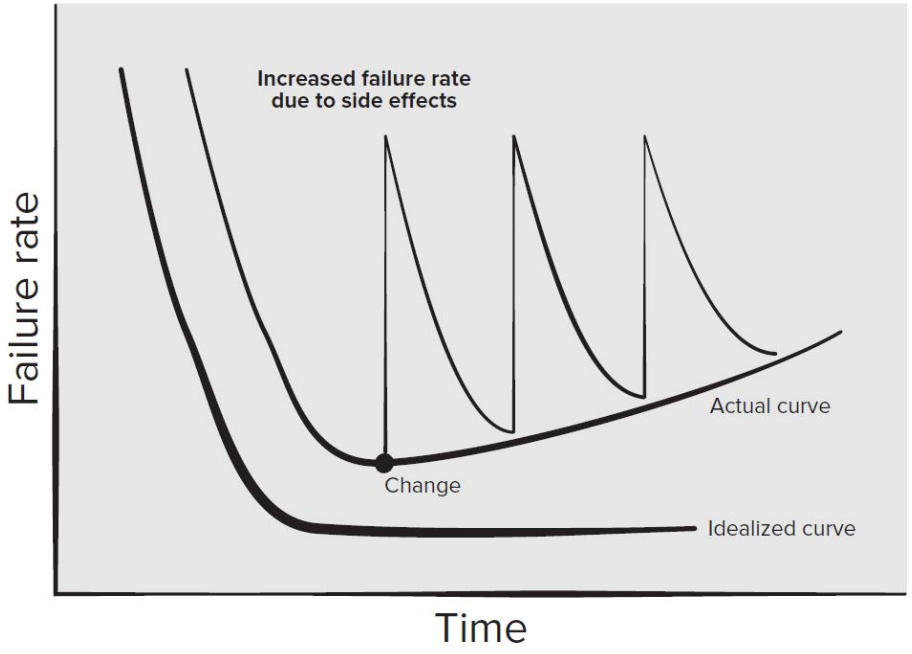
Software Evolution

Hardware will be worn - Bathtub curve (left)

Software will be deteriorated(退化) - Sail curve (right)

Question: Software will be evolution, not involution. Why?





12.1 Characteristics of Software

The marginal cost of software is Zero

Marginal cost also called incremental cost, refers to the cost to be paid for each additional unit of product production.

The marginal cost of software is Zero.

The source code development cost of the first software product is high, but the cost of copying the source code is close to zero, and the quality of the copy is the same as that of the first product. Example: Open source software.

12.1 Characteristics of Software

Network Effect

The utility that a given user derives from the good depends upon the number of other users who are in the same network as he or she. That means the larger the network, the more the users benefit.

Network effects are either direct effects and/or indirect effects.

Direct network effects arise from the fact that by employing the same software standards or common technologies, users can communicate with each other more simply and therefore more cost-effectively.

Example of a direct network effect is the Android Ecosystem.

- Google's Android ecosystem have had significant success in getting existing software companies as well as new start-up ventures to offer software products and services within them.

12.1 Characteristics of Software

Network Effect

Indirect network effects, by contrast, result from the dependency between consumption of a basic good and consumption of complementary goods and services. They arise, therefore, when the wider adoption of a good generates a broader range of associated goods and services, which enhances the utility of the basic good.

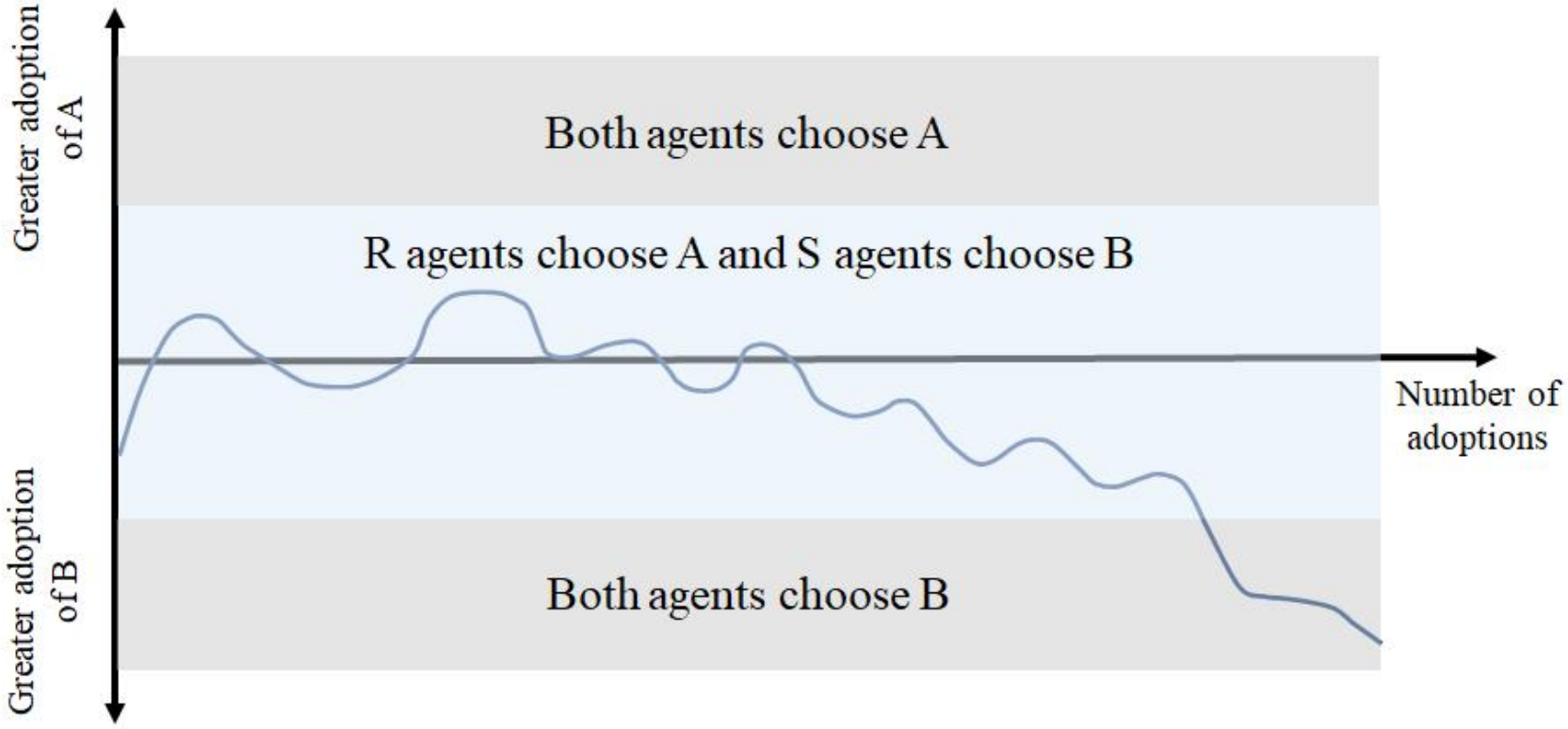
Indirect network effects occur in the context of standard software and complementary consulting services, operating systems with compatible application software, or with regard to the availability of programming language experts and/or tools.

Network effects lead to demand-sided economies of scale and to what is known as positive feedback respectively, increasing returns. Shapiro and Varian summarize this phenomenon as follows: "***Positive feedback*** ***makes the strong get stronger and the weak get weaker***''.

Figure in next page shows this self-reinforcing cycle both for direct and for indirect network effects.

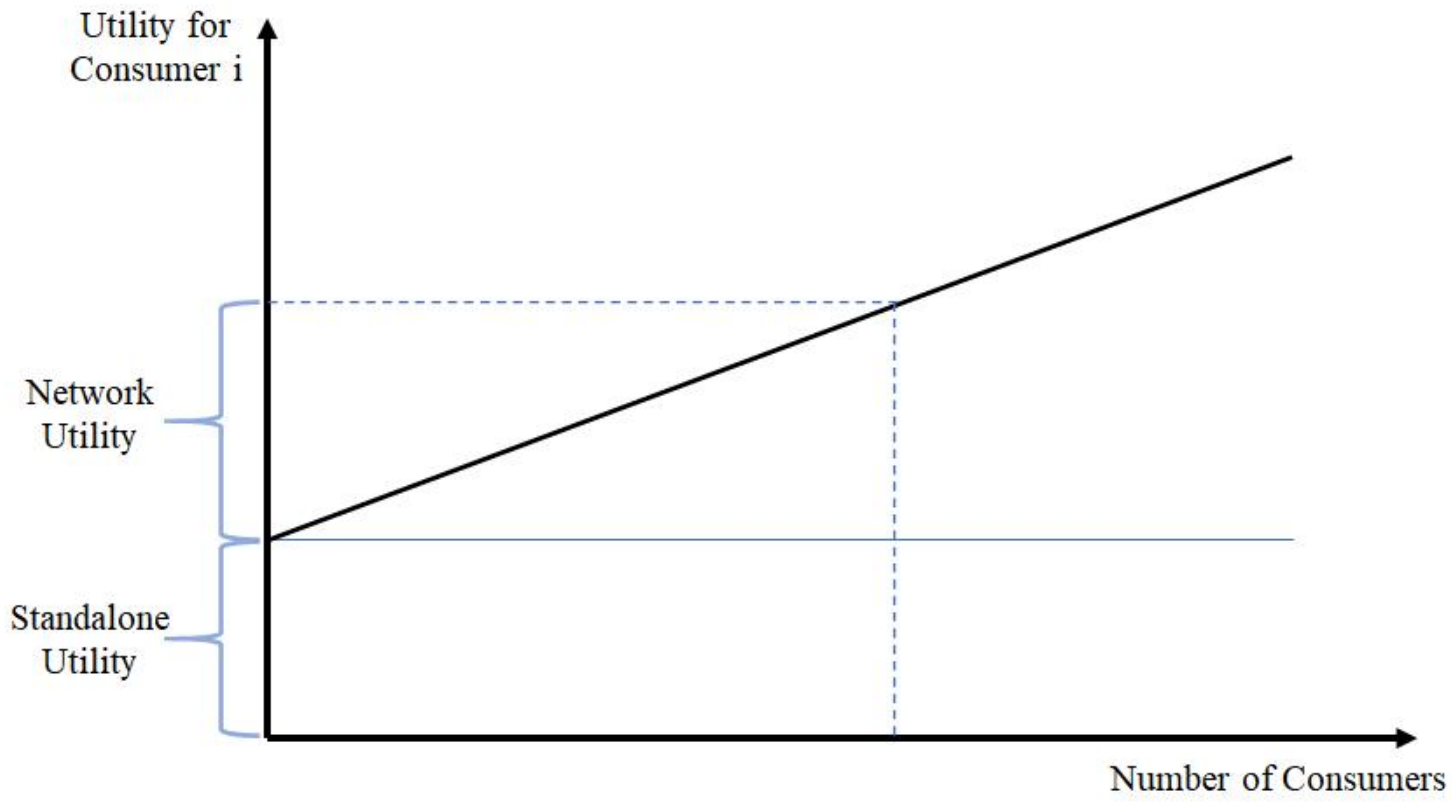
12.1 Characteristics of Software

Case：Network Effect



12.1 Characteristics of Software

- Case：Network Effect Utility(效用)



12.1 Characteristics of Software

Externality (外部性)

Software has **externality**.

The value of software product to consumers increases with the number of other users using the software or the software platform.

Examples:

- Taobao platform, as a software carrier, entry of the new buyers or sellers, for existing Taobao users, free of charge to increase the new value of the transaction, without paying for this part of the value.
- Wechat, if a few people use Wechat, its value is relatively low, but if most people use Wechat, its value is much greater. The benefits of those who use this communication mode earlier will increase with the increase of Wechat users, because they can contact more people through Wechat.

12.1 Characteristics of Software

Ecosystem

Ecosystem in Market terminology refers to the inter-dependence between demand and supply.

In the Android ecosystem this translates to inter-dependence between users, developers, and equipment makers. One cannot exist without the other:

- Android users, buy devices and applications.
- Equipment makers, sell devices, sometimes bundled with applications.
- Developers, buy devices, then make and sell applications.

Android User

Android users have more space for customizability for their android devices. Android users are smarter than other users and they are perceived to have greater levels of support. Android users are also more likely to prefer saving their cost and love the openness of the platform also they like to customize their device. Android users are fancier to prefer saving money and also android user like customizing their android handset/device.

12.1 Characteristics of Software

Roles in Ecosystem

Developers

Android Developers are the professional software developer in designing applications as well as developing applications for Android. Some of the following tasks where an android developer can play his role in the development of android apps:

Design and build advanced applications for the android platform. Collaborate and define with development teams for design and deliver new cool features. Troubleshoot and fix bugs in new and existing applications for Users.

Evaluate and implement new development tools to work with outside data sources and APIs.

12.1 Characteristics of Software

Roles in Ecosystem

Equipment Maker

Smartwatches: A smartwatch is a handheld, wearable device that closely relates a wristwatch or other time device. In addition to telling time, many smartwatches are wireless connectivity oriented such as Bluetooth capable. The traditional watch becomes, in effect, a wireless Bluetooth technology extending the capabilities of the wearer's smartphone to the watch.

Smart TV: An Android TV box is a small computer that plugs into any TV and gives the user the ability to stream content, locally and online. Apps can be downloaded from the Google Play Store, installed, and do most anything a standard computer can do from streaming videos to writing an email.

Smart Speakers: Smark speakers are booming in the market now, Smark speakers like Google Home, Alexa, We can control our android device via voice using these smart speakers.

E-Reader: E-Reader is a device used for reading e-books, digital newspapers, other reading stuff.

12.2 Sources of Software Value

■ Software Value Model – DIKW model

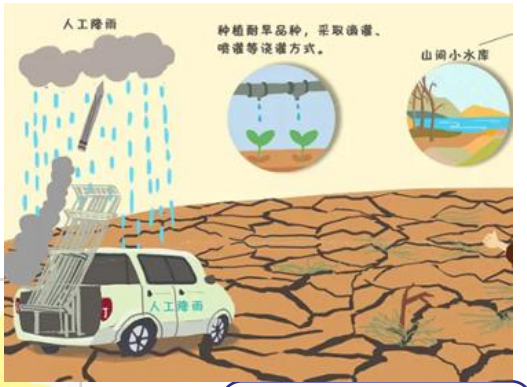
The objects processed by software include data, information, knowledge, and intelligence. In terms of whether it includes wisdom, I believe that wisdom is a unique human ability with creative meaning, and software currently does not generate wisdom. Software value derives from processing data and information, even from generating knowledge.

Example: Decision on whether to bring an umbrella during the rainy season.

- Intelligence: based on the probability of the weather forecast.
- Wisdom: Waterproof windbreak.

12.2 Sources of Software Value

- Case study: Sources of software value



Decision

Knowledge

Information

Data

12.2 Sources of Software Value

Sources of software value

Philosophers only explain the world in different ways, and the problem lies in changing the world. - On Feuerbach‘s outline, Karl Marx, 1845.

The value of software comes from its activities of processing data, information, knowledge, even providing services of artificial intelligence for its end-users.

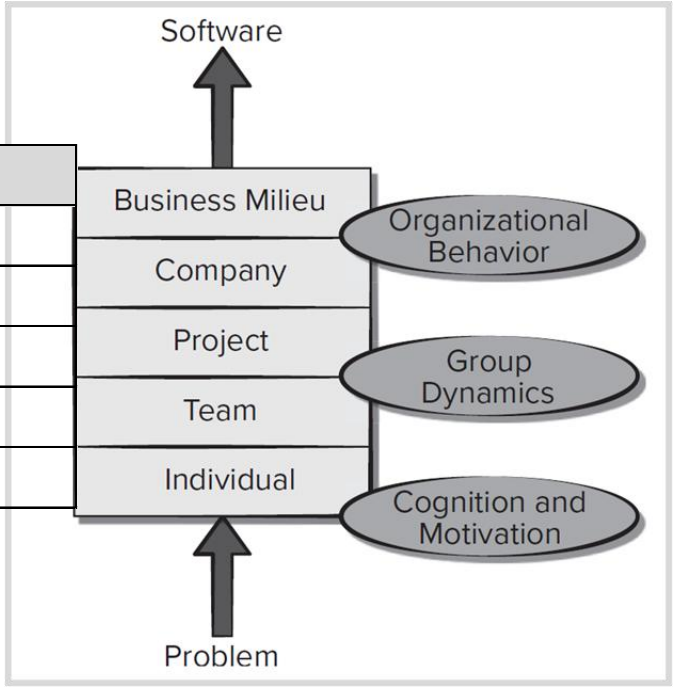
Question: please give some examples of sources of software value?

12.3 Viewpoints of SEE Research

■ Different viewpoints on software value

Different economic entities have different wants & viewpoints on software value and its assessment.

Economic Entities	Focus(es)
Society	Industy, Environment
Enterprise	Business value, Organization
Project	Effect, Efficiency, Output
Team	Software process & management
Individual	Design & Implementation



12.3 Viewpoints of SEE Research

Case: On Software Quality

Software quality is the core issue of software engineering research and the ultimate goal pursued by software engineering economics.

The following views on software quality:

A priori view(先验论): quality is recognizable, but undefined.

User view: quality is just to achieve the goal.

Developer's point of view: quality is the consistency between products and specifications.

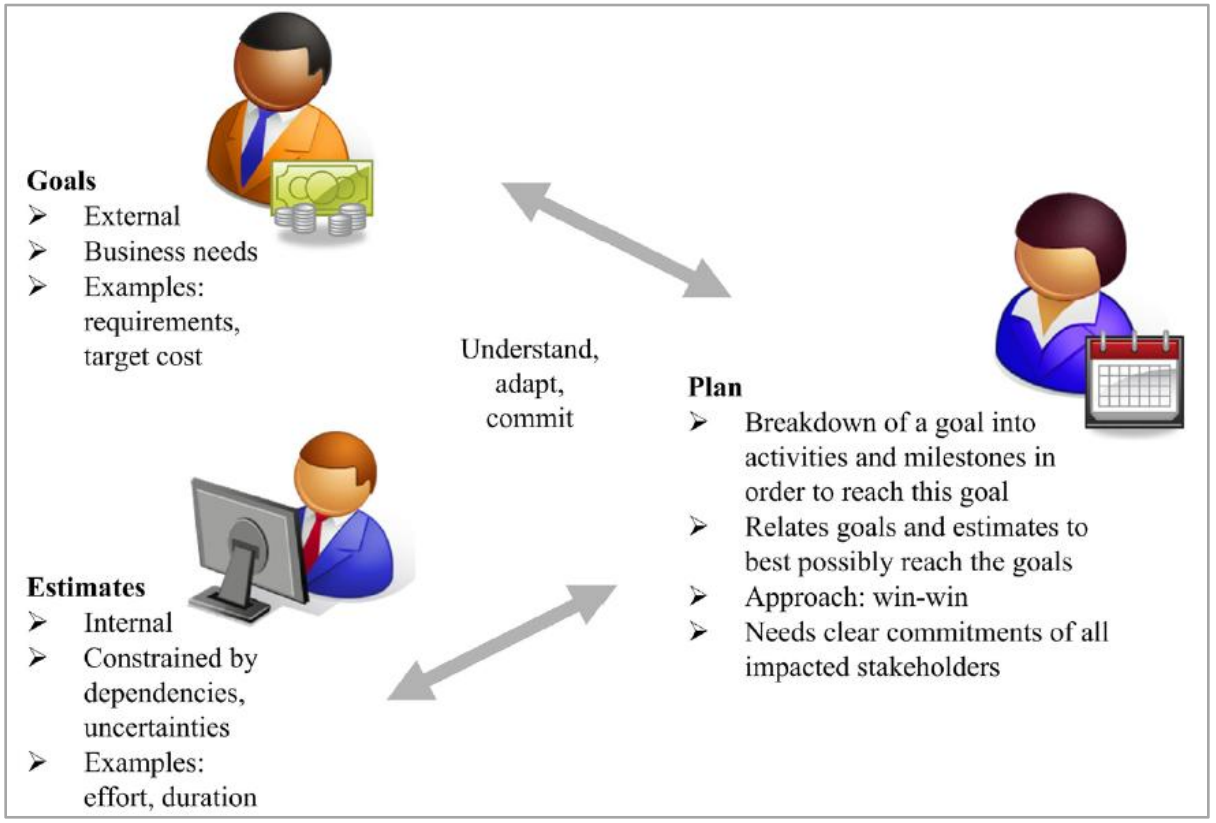
Product view: quality is related to the internal characteristics of products.

Value based view: quality depends on the amount the customer is willing to pay.

12.4 Paradigm of SEE Analysis

Goal-Plan-Metrics in balance

Goals in software engineering economics are mostly business goals or business objectives. A business goal relates business needs (such as increasing profitability) to investing resources (such as starting a project or launching a product with a given budget, content, and timing).



12.4 Paradigm of SEE Analysis

■ Goal-Plan-Metrics in balance

Goals apply to operational planning, for instance, to reach a certain milestone at a given date or to extend software testing by some time to achieve a desired quality level, and to the strategic level (such as reaching a certain profitability or market share in a stated time period).

Metrics are a well-founded evaluation of resources and time that will be needed to achieve stated goals, e.g. Effort, Schedule, Cost Estimation and Maintenance Cost Estimation.

Estimates should not be driven exclusively by the project goals because this could make an estimate overly optimistic. Estimation is a periodic activity, should be continually revised during a project.

12.4 Paradigm of SEE Analysis

■ Goal-Plan-Metrics in balance

Plan describes the activities and milestones that are necessary in order to reach the goals of a project. The plan should be in line with the goal and the estimate, which is not necessarily easy and obvious, such as when a software project with given requirements would take longer than the target date foreseen by the client. In such cases, plans demand a review of initial goals as well as estimates and the underlying uncertainties and inaccuracies. Creative solutions with the underlying rationale of achieving a win-win position are applied to resolve conflicts.

To be of value, planning should involve consideration of the project constraints and commitments to stakeholders. Figure above shows how goals are initially defined. Estimates are done based on the initial goals. The plan tries to match the goals and the estimates. This is an iterative process, because an initial estimate typically does not meet the initial goals.

12.5 Business Models of SEE

- Business Model

12.5 Business Models of SEE

Business Model

The customer: Who are our target customers? It is important that you understand precisely which customer segments are relevant for you and which ones you will and won't address with your business model. Customers are at the very heart of every business model – always! There are no exceptions.

The value proposition: What do we offer to customers? This second dimension defines your company's offerings (products and services) and describes how you cater for your target customers' needs.

The value chain: How do we produce our offerings? In order to put your value proposition into effect you need to carry through various processes and activities. These processes and activities in conjunction with related resources and capabilities and their coordination along the company's value chain make up the third dimension of business model design.

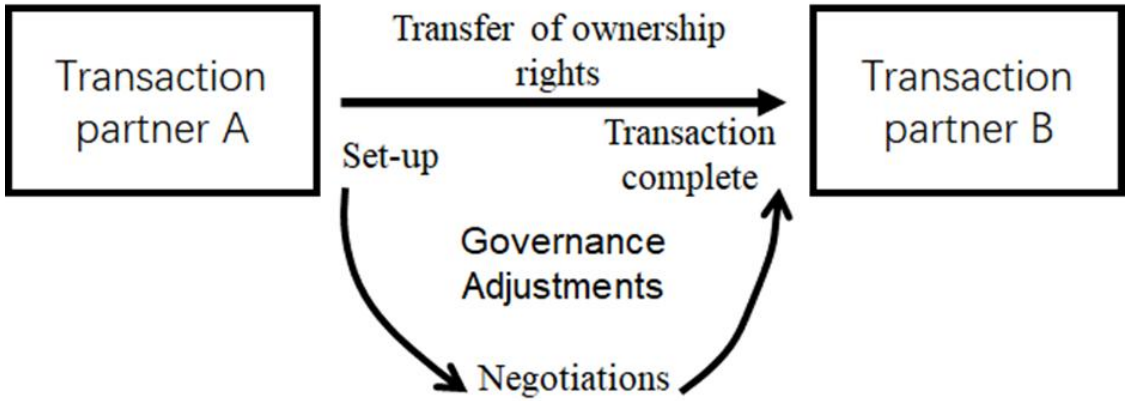
The profit mechanism: Why does it generate profit? This fourth dimension, which includes aspects such as cost structures and revenue-generating mechanisms, clarifies what it is that makes a business model financially viable. It provides an answer to the central question that every company needs to ask: how do we produce value for our shareholders and stakeholders? Or simpler: why does the business model work commercially?

12.5 Business Models of SEE

Case: Make-or-Buy Decision Business Model

The position a company assumes within the value chain is of great importance. In essence, this is a classic make-or-buy decision: what should the company produce or develop in-house? what should it source from third parties?

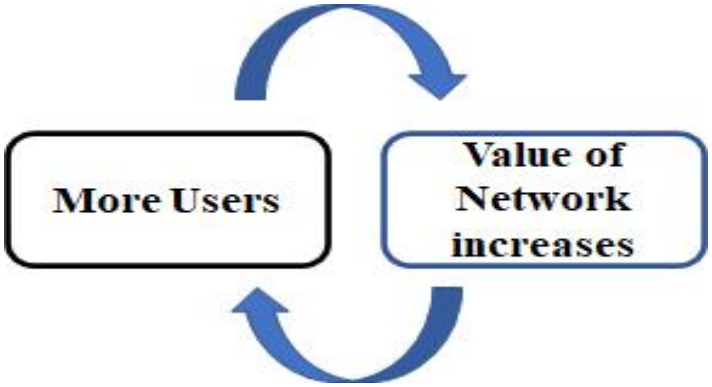
Comparation between Software value and Transaction cost provides some interesting insights into these question.



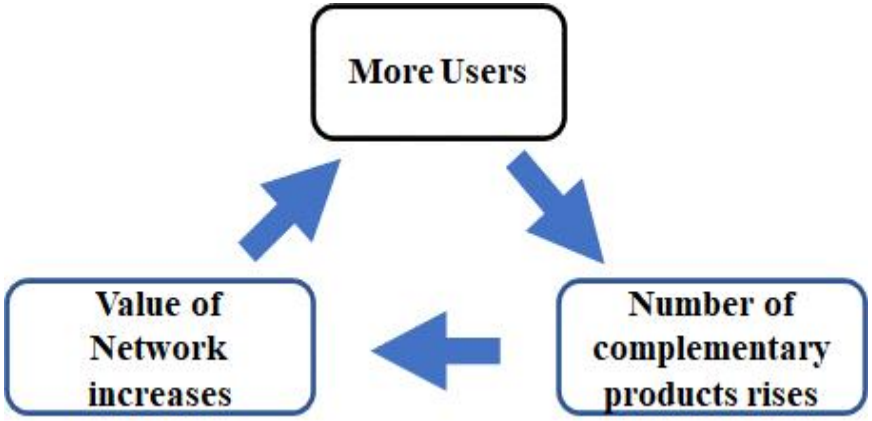
Factors influencing costs	Key assumptions
• Specificity of investment • Uncertainty of requirement • Frequency of transaction	• Benefit-maximizing behavior • Bounded rationality • Risk neutrality

12.5 Business Models of SEE

Case: Network Effect Business Model



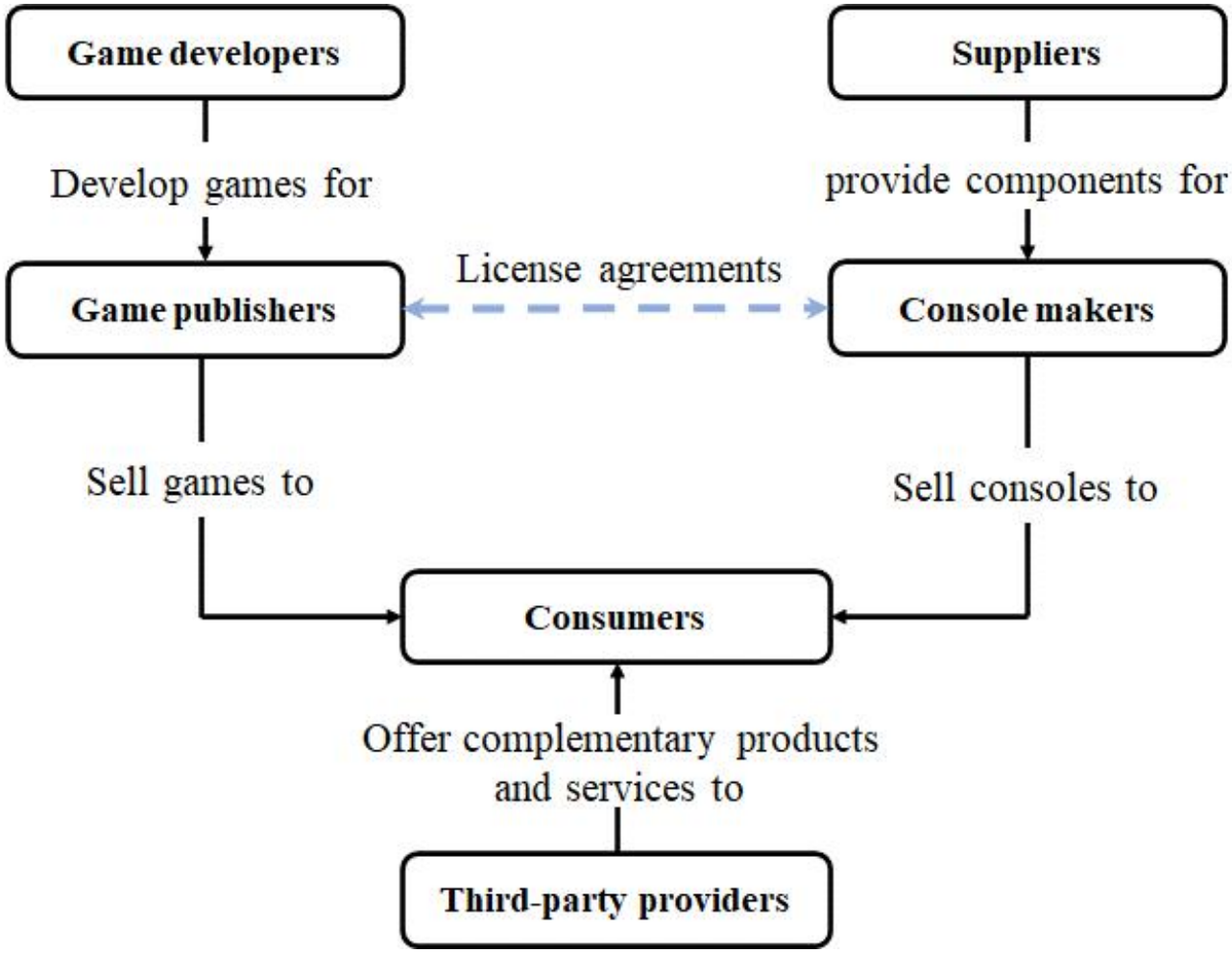
Direct network effect



Indirect network effect

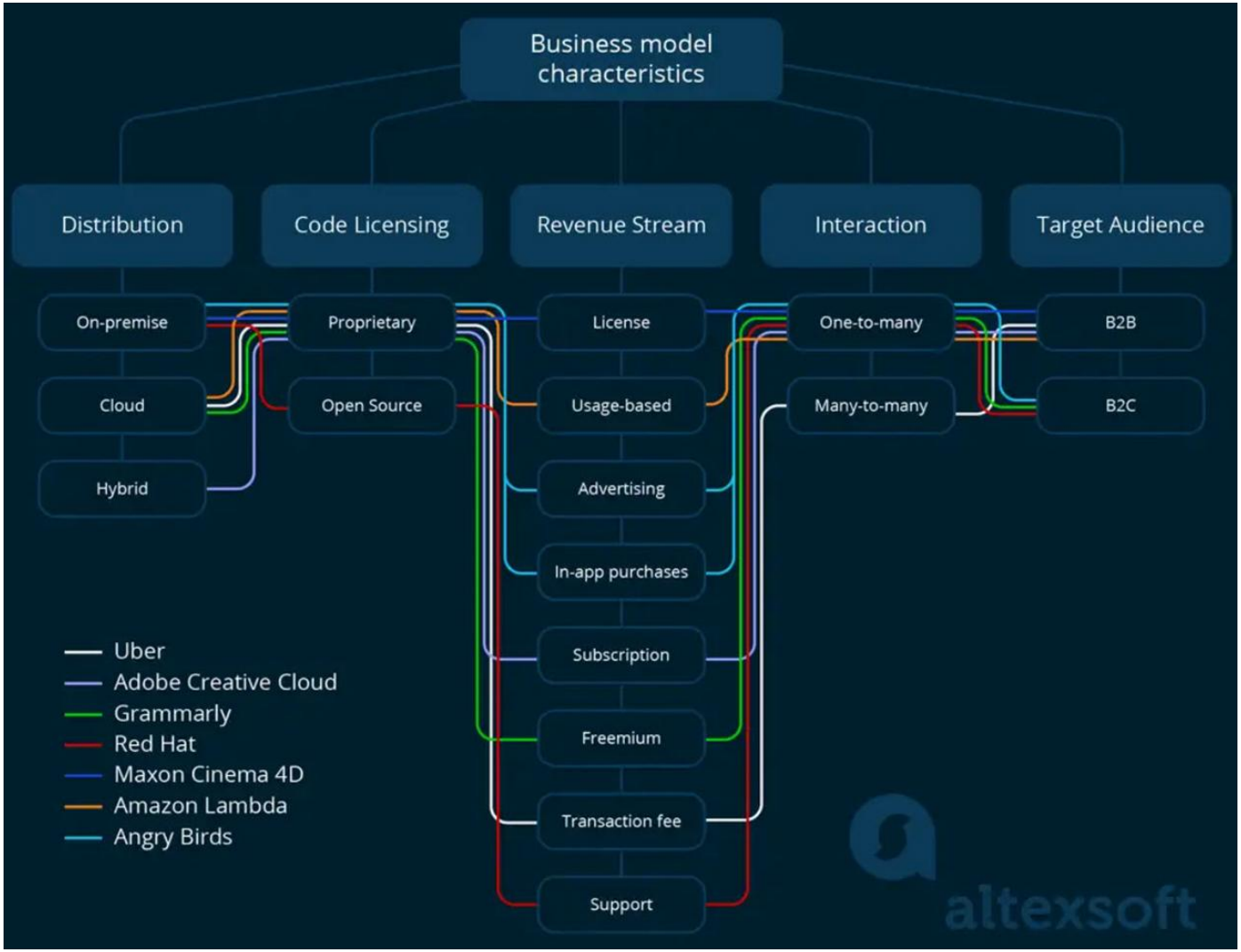
12.5 Business Models of SEE

Case: Game Software Business Model



12.5 Business Models of SEE

- Typical Business Models in Software Industry



12.5 Business Models of SEE

Typical Business Models in Software Industry

Category	What it describes	Types of software business models
Distribution	How and where you provide products to customers	1. On-premise 2. Cloud-based 3. Hybrid
Source code licensing	Who has access to the source code and what they are allowed to do with it	1. Proprietary 2. Open-source
Revenue streams	How your company makes money	• Ads • Sales • Subscriptions • Services • Combinations
Target audience	Who you are selling to	1. B2B 2. B2C

12.6 Law of Software Demand & Supply

Davidow's Law

Davido's law holds that in the network economy, the **First Generation** of products entering the market can automatically obtain 50% of the market share.

Therefore, if an enterprise wants to occupy a dominant position in the market, it must be **the First** to develop a new generation of products. It's better to be the first one than the second or the third, even though your product was not perfect at that time.

The law also holds that any enterprise in the industry must be the first to eliminate their own products, that is, to update their products as soon as possible, rather than let the fierce competition eliminate your products. This is actually an inevitable result of living in "Internet time".

When William Davido was vice president of Intel, he noticed the importance of improving the speed of product update, and put forward this law.

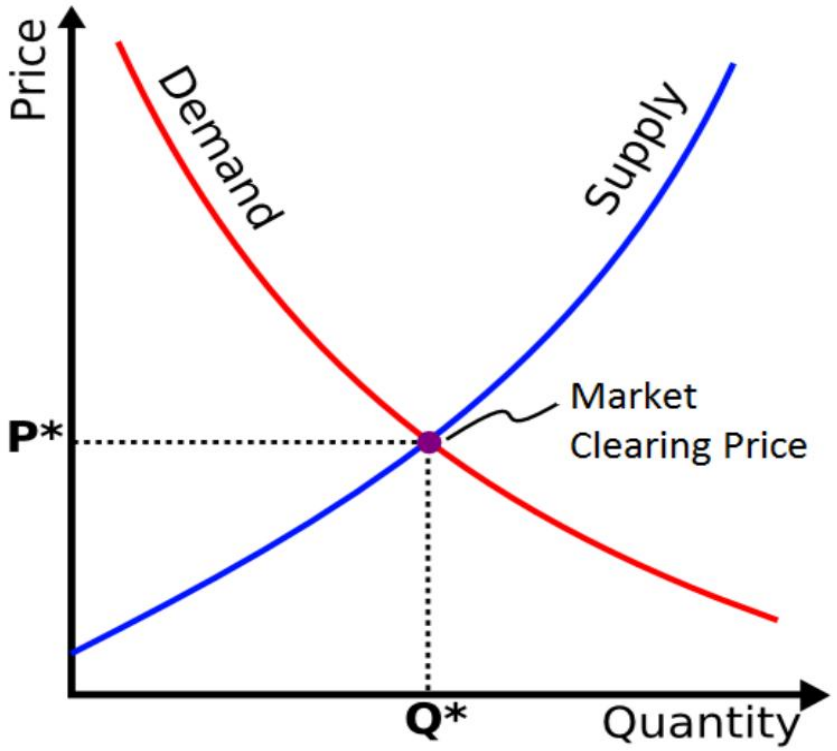
Insight: Time to market.

12.6 Law of Software Demand & Supply

Law of Demand & Supply

The law of supply and demand predicts that if the supply of goods or services outstrips demand, prices will fall. If demand exceeds supply, prices will rise.

In a free market, the equilibrium price is the price at which the supply exactly matches the demand.



12.6 Experiment-in-class: Law of Demand & Supply

■ Experiment: Discover Demand Curve by Auction

Please read the experimental guidebook and complete the experiment according to the requirements of the guidebook.

Step1: The teacher explains the requirements of this experiment.

Step2: Faced with auction items, each student provides hie/her own suitable price.

Step3: Fill in your bid on a small card, indicate your student ID and name clearly.

Step4: Two student representatives will publicly vote.

Step5: The teacher calculate the required prices and quantities

Step6: The teacher generates **the demand curve** for this item auction.

Step7: The teacher displays **the best price** of this auction, announce the lucky one.

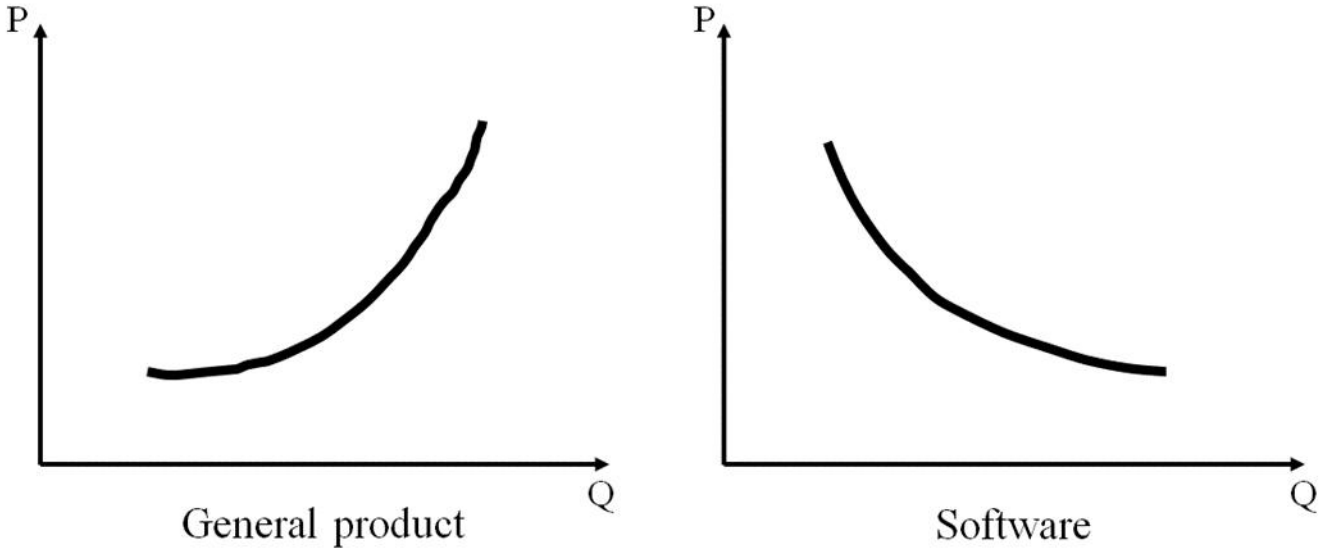
Step8: After class, students should complete an experimental report according to the requirements of experimental guidebook, including but not limited record their own quotation and the best price of this auction, submit the report on Canvas.

12.6 Law of Software Demand & Supply

Law of Supply of Software

Because of the increasing marginal cost, the supply curve of general products inclines upward, that is, when the product price is higher, the more products the manufacturer is willing to provide.

Software products are characterized by high fixed cost and low marginal cost. With the increase of network scale, the price of software products decreases, and the supply curve shows a downward trend.

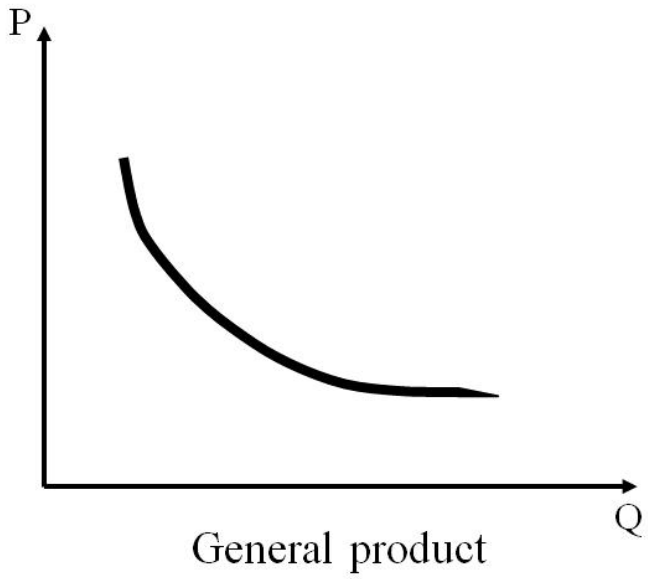


12.6 Law of Software Demand & Supply

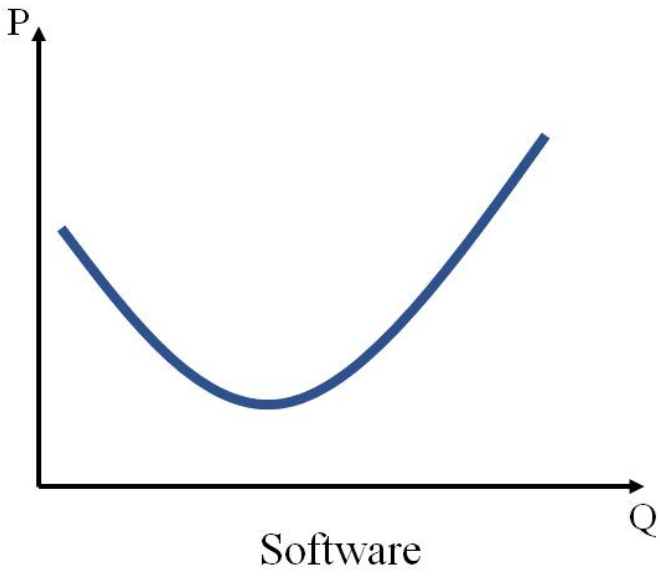
Law of Demand of Software

The lower the price is, the more products consumers are willing to buy.

The reason is the law of diminishing marginal utility. On the one hand, the demand of software products follows the law of diminishing marginal effect, and the price tends to decrease with the increase of demand; On the other hand, the more consumers join the network, the greater the value of the network, and the greater the consumers' willingness to pay.



General product



Software

12.6 Law of Software Demand & Supply

Law of Demand of Software

Therefore, the demand curve of software products shows a downward slope in the left half of the curve, which is the same as that of general products. With the increase of demand, consumers' willingness to pay decreases. At this time, the diminishing marginal utility effect is greater than the network effect.

In the right half of the curve, because the network effect is greater than the marginal diminishing effect, the demand curve inclines upward. With the expansion of the network scale, the value of the network increases, and consumers' willingness to pay increases accordingly.

12.7 Case Study: Software Business Model

- Scientific American System

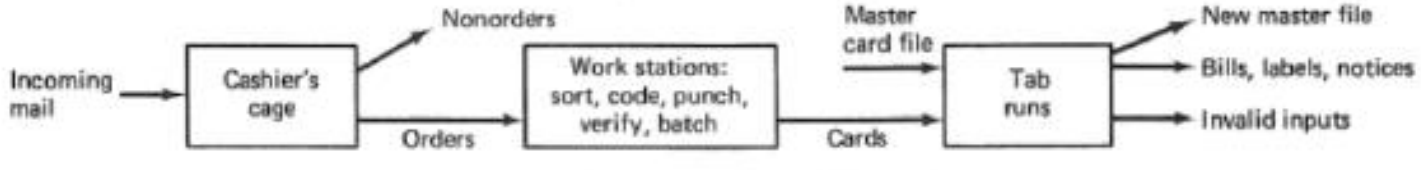


FIGURE 1-1 *Scientific American:* The old system

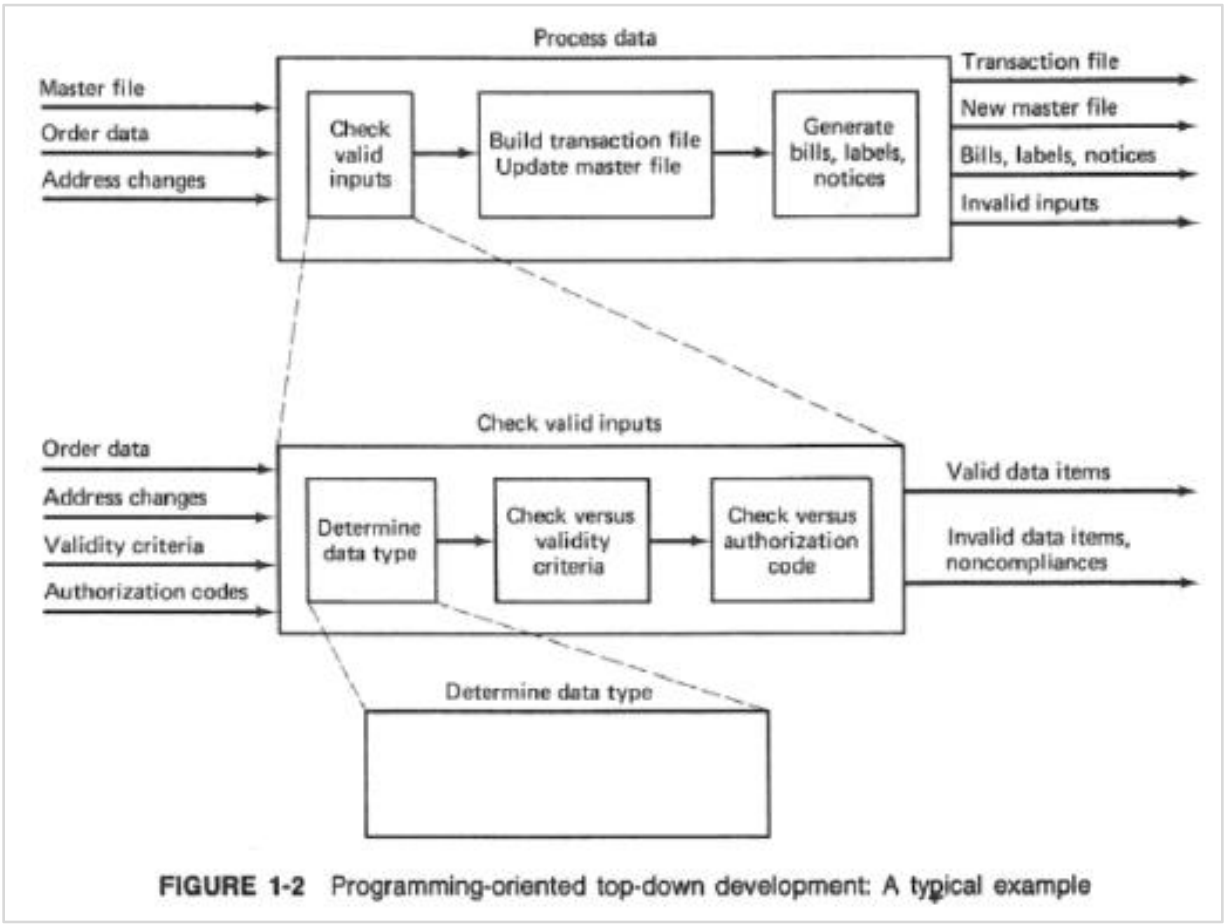


FIGURE 1-2 Programming-oriented top-down development: A typical example

12.7 Case Study: Software Business Model

- Scientific American System

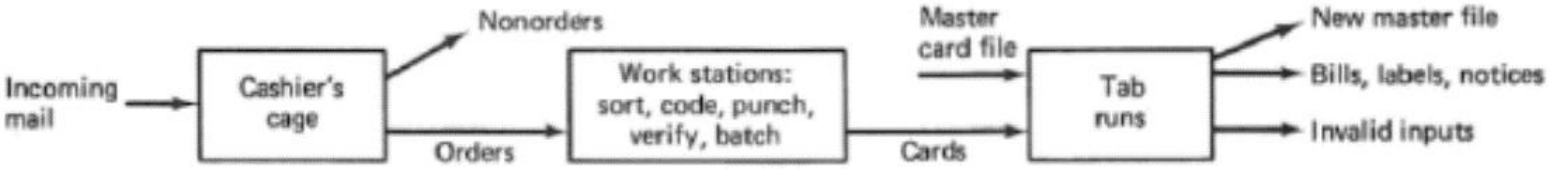


FIGURE 1-1 *Scientific American:* The old system

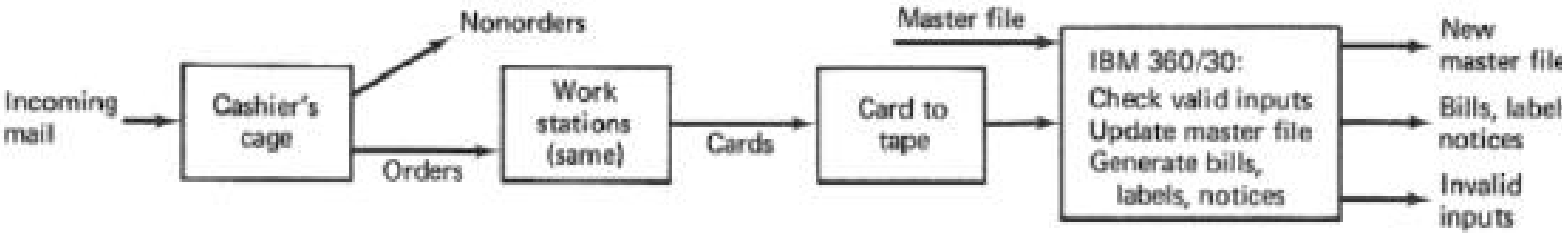


FIGURE 1-3 *Scientific American:* The programming solution

12.7 Case Study: Software Business Model

Scientific American System

The main results were

Costs went up
Reliability and quality of service went down
More clerical people were required
Employee moral went down
Employee turnover went up

The main reason for these results was that the programming solution overlooked some key operational elements of the problem. The overlooked factors contained significant performance and economic implications.

For example, trivial input errors would cause the program to reject entire batches of input, or cause a complete stop of an entire update run. When this happened, the transaction input listing had to be found and manually checked for errors. Correcting these errors involved another cumbersome process of updating the transaction file. In the old tab run system, these errors were usually discovered and fixed on the spot by the operators, who had extensive experience on errors to expect and how to fix them.

12.7 Case Study: Software Business Model

Scientific American System

In economic programming approach, considering the following questions

1. What objectives is the user trying to satisfy?
2. What decisions do we control which affect these objectives?
3. What items dictate constraints on our range of choices?
4. What criteria should we use to evaluate candidate solutions? How are the criterion values related to the decision variables involved in candidate solutions?
5. What decision provides us with the most satisfactory outcome with respect to the criteria we have established?

1.What objectives is the user trying to satisfy?

For Scientific American, the primary objectives were to increase subscription fulfillment speed and reliability, and to reduce costs, staff level and turnover, and customer complaints. Answering this question also involved gathering and analyzing data on primary sources of costs, errors, delays and frustrations.

2.What decisions do we control which affect these objectives?

For Scientific American, these decisions include not only computer decisions, but also decisions to use separate postal boxes for different types of orders, neatly transferring the job of sorting to the U.S. Postal Service.

12.7 Case Study: Software Business Model

Scientific American System

The result at Scientific American was to install five minicomputer-based intelligent terminals, at $10,000 each, to eliminate the IBM 360/30 at a saving of roughly $9,000 per month, and to obtain service bureau processing at roughly $7,000 per month. The resulting system required significantly fewer people, so that Scientific American was able to amortize their investment in the terminals by the end of the first year of operation.

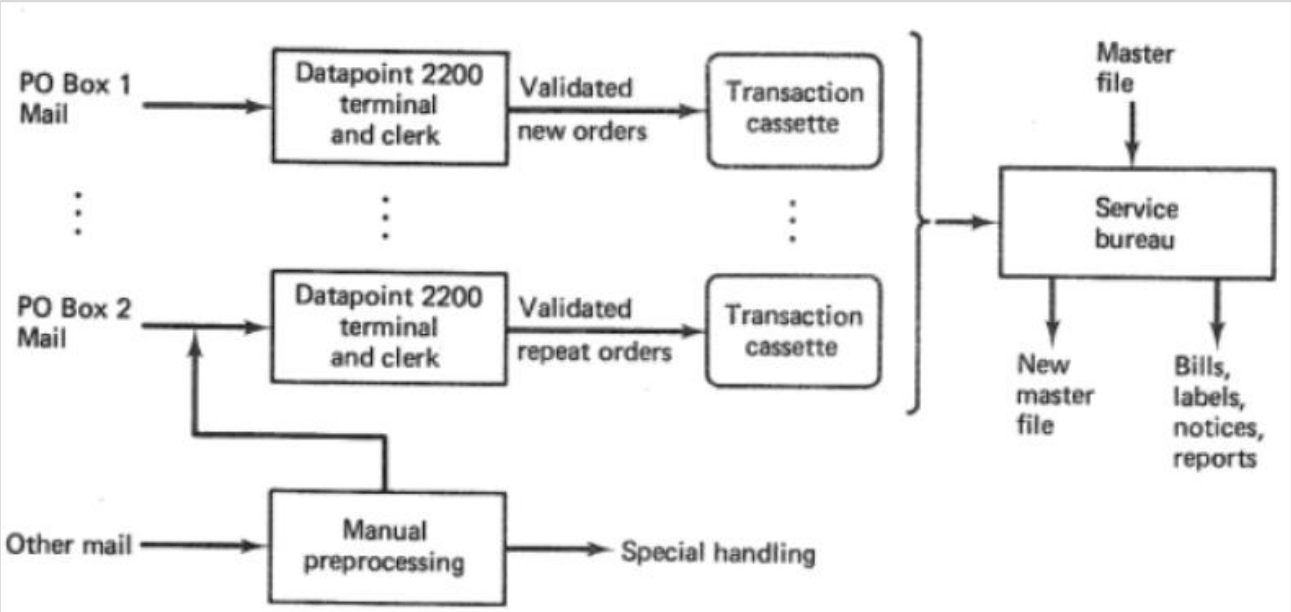


FIGURE 1-4 *Scientific American:* The economic–programming solution

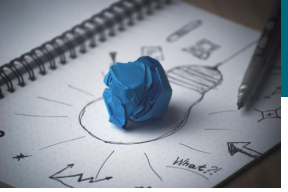
12.7 Case Study: Software Business Model

Scientific American System

At the end of the year the results were as follows:

- The number of clerical workers was reduced from over 40 to fewer than 20.
- Clerical errors were reduced to a small fraction of the number of unavoidable "subscriber errors", for example, duplicate orders or payments.
- Virtually all transactions were handled in less than a week, compared to frequent delays of over a month in the previous system.
- The new system was handling a 33 percent increase in workload.
- Subjective responses from the clerks indicated an increased feeling of job satisfaction. The single-step order entry gave them a more meaningful job to perform, with more opportunity to exercise their judgement in handling problems and exceptions.

The economic programming approach lad to a solution meeting or exceeding all of Scientific American's improvement objectives, where the pure programming solution had only succeeded in making things worse for Scientific American.



Core concepts & Assignments

Core Concepts

Characteristics of Software
Software Value & its Sources
Viewpoints of SEE Research
Paradigm of SEE Analysis
Business Models of SEE
Law of Software Demand & Supply

Assignment

Join & Learn MOOC of Software Engineering Economics in Wisdom Tree(智慧树) platform, the linkage is https://coursehome.zhihuishu.com/courseHome/1000050734#teachTeam